

Thuật toán đường đi ngắn nhất (Shortest Path Algorithms)

Giảng viên: Trần Đình Tân

Môn: Thuật toán ứng dụng

Sau bài học, người học có thể:

- Phát biểu chính xác bài toán đường đi ngắn nhất một nguồn và mọi cặp đỉnh.
- Giải thích thao tác nối lỏng cạnh và vai trò của mảng truy vết đường đi.
- Chọn đúng BFS, Dijkstra, Bellman–Ford hoặc Floyd–Warshall theo giả định về trọng số cạnh.
- Phân tích độ phức tạp thời gian và bộ nhớ của từng thuật toán.
- Nhận diện các trường hợp biên: đỉnh không tới được, cạnh âm, chu trình âm, và nhiều đường đi tối ưu.

Nội dung

- 1 Bài toán
- 2 BFS và Dijkstra
- 3 Bellman–Ford
- 4 Floyd–Warshall
- 5 So sánh và thực hành

Mô hình đồ thị có trọng số

Mô hình

Cho đồ thị $G = (V, E)$ và hàm trọng số $w : E \rightarrow \mathbb{R}$. Với đường đi $P = (v_0, v_1, \dots, v_k)$, độ dài của P là

$$w(P) = \sum_{i=1}^k w(v_{i-1}, v_i).$$

- V : tập đỉnh; E : tập cạnh. Đồ thị có thể có hướng hoặc vô hướng.
- Trọng số có thể biểu diễn khoảng cách, thời gian, chi phí hoặc rủi ro.
- Cần xác định rõ giả định về trọng số trước khi chọn thuật toán.

Các biến thể của bài toán

- **Một nguồn – mọi đích** (single-source shortest paths): từ nguồn s đến mọi $v \in V$.
- **Một cặp đỉnh** (single-pair): từ nguồn s đến đích t .
- **Mọi cặp đỉnh** (all-pairs shortest paths): tính khoảng cách giữa mọi cặp (i, j) .

Đầu ra chuẩn

Thường cần hai phần: giá trị khoảng cách $\text{dist}(s, v)$ và mảng truy vết $\text{pred}[v]$ để khôi phục đường đi.

Khi nào khoảng cách ngắn nhất tồn tại?

- Nếu v không tới được từ s , đặt $\text{dist}(s, v) = \infty$.
- Nếu tồn tại chu trình âm đi tới được từ s và từ đó đi tiếp tới v , thì không có khoảng cách ngắn nhất hữu hạn đến v .
- Với đồ thị vô hướng, một cạnh âm thường tạo chu trình âm độ dài hai nếu được phép đi qua lại cùng cạnh.

Lưu ý quan trọng

“Có cạnh âm” không tự động làm bài toán sai; điều làm mất nghiệm hữu hạn là **chu trình âm có thể ảnh hưởng tới đích cần xét**.

Thao tác nối lỏng cạnh

Relaxation

Với cạnh (u, v) , nếu

$$d[v] > d[u] + w(u, v),$$

thì cập nhật

$$d[v] \leftarrow d[u] + w(u, v), \quad \text{pred}[v] \leftarrow u.$$

- $d[v]$ là khoảng cách tạm thời tốt nhất đã biết tới v .
- Nối lỏng cạnh là thao tác lõi của Dijkstra và Bellman–Ford.
- Mảng pred cho phép lần ngược từ đích về nguồn để lấy đường đi.

Trường hợp không trọng số: BFS

Kết luận chính xác

BFS cho đường đi ngắn nhất khi mọi cạnh có cùng trọng số đơn vị, tức là mỗi cạnh có chi phí bằng 1.

- BFS duyệt theo từng lớp khoảng cách: lớp 0, lớp 1, lớp 2, v.v.
- Độ phức tạp với danh sách kề: $O(|V| + |E|)$.
- Nếu trọng số không đồng nhất, BFS không còn tối ưu theo tổng trọng số.

Bảng chạy BFS theo từng vòng lặp

Ví dụ đồ thị không trọng số: $A \rightarrow B$, $A \rightarrow C$, $B \rightarrow D$, $C \rightarrow E$, $D \rightarrow F$, $E \rightarrow F$. Nguồn: A .

Bước	Lấy ra	Đỉnh mới phát hiện	Hàng đợi sau bước	Khoảng cách mới biết
0	–	A	$[A]$	$d[A] = 0$
1	A	B, C	$[B, C]$	$d[B] = 1, d[C] = 1$
2	B	D	$[C, D]$	$d[D] = 2$
3	C	E	$[D, E]$	$d[E] = 2$
4	D	F	$[E, F]$	$d[F] = 3$
5	E	Không có	$[F]$	Không đổi
6	F	Không có	$[]$	Không đổi

Kết quả cuối: $d[A] = 0$, $d[B] = d[C] = 1$, $d[D] = d[E] = 2$, $d[F] = 3$.

Ý chính

Dijkstra luôn chọn đỉnh chưa chốt có khoảng cách tạm thời nhỏ nhất, sau đó nối lỏng các cạnh đi ra từ đỉnh đó.

- Thuật toán có bản chất tham lam.
- Đúng khi mọi cạnh có trọng số không âm: $w(e) \geq 0$ với mọi $e \in E$.
- Sau khi một đỉnh u được chốt, giá trị $d[u]$ chính là khoảng cách ngắn nhất từ nguồn đến u .

Bất biến đúng của Dijkstra

Bất biến

Tại mỗi bước, với mọi đỉnh đã chốt u , ta có $d[u] = \text{dist}(s, u)$.

- Điều kiện $w(e) \geq 0$ bảo đảm rằng một đường đi đi qua các đỉnh chưa chốt không thể làm giảm khoảng cách của đỉnh đã chốt.
- Vì vậy, đỉnh chưa chốt có d nhỏ nhất có thể được chốt an toàn.
- Nếu có cạnh âm, lập luận này bị phá vỡ; Dijkstra có thể trả kết quả sai.

Pseudocode Dijkstra

- ❶ Khởi tạo $d[s] = 0$, $d[v] = \infty$ với mọi $v \neq s$; đặt $\text{pred}[v] = \text{None}$.
- ❷ Đưa các đỉnh vào hàng đợi ưu tiên theo khóa $d[v]$.
- ❸ Lặp cho đến khi hàng đợi rỗng:
 - ❶ Lấy ra đỉnh u có $d[u]$ nhỏ nhất.
 - ❷ Nếu u đã được xử lý trước đó, bỏ qua bản ghi cũ trong hàng đợi.
 - ❸ Với mỗi cạnh (u, v) , thực hiện nới lỏng cạnh.
- ❹ Nếu chỉ cần đường đi từ s đến t , có thể dừng khi t được chốt.

Độ phức tạp của Dijkstra

- Cài đặt bằng mảng và ma trận kề: $O(|V|^2)$.
- Cài đặt bằng danh sách kề và heap nhị phân: $O((|V| + |E|) \log |V|)$.
- Với đồ thị liên thông thưa, thường viết gọn là $O(|E| \log |V|)$.
- Bộ nhớ phụ thường là $O(|V| + |E|)$ nếu lưu danh sách kề, d , $pred$ và hàng đợi ưu tiên.

Điểm cần tránh

Không dùng Dijkstra cho đồ thị có cạnh âm nếu không có biến đổi hoặc chứng minh bổ sung.

Ví dụ chạy tay Dijkstra

Xét đồ thị có hướng với các cạnh:

$$A \rightarrow B : 4, \quad A \rightarrow C : 2, \quad C \rightarrow B : 1, \quad B \rightarrow D : 5,$$

$$C \rightarrow D : 8, \quad C \rightarrow E : 10, \quad D \rightarrow E : 2, \quad D \rightarrow F : 6, \quad E \rightarrow F : 3.$$

Kết quả từ nguồn A

$$d[A] = 0, \quad d[C] = 2, \quad d[B] = 3, \quad d[D] = 8, \quad d[E] = 10, \quad d[F] = 13.$$

Một đường đi ngắn nhất từ A đến F là

$$A \rightarrow C \rightarrow B \rightarrow D \rightarrow E \rightarrow F.$$

Bảng chạy Dijkstra theo từng vòng lặp

Với ví dụ từ nguồn A , ký hiệu " ∞ " là chưa biết đường đi hữu hạn.

Bước	Đỉnh chốt	$d[A]$	$d[B]$	$d[C]$	$d[D]$	$d[E]$	$d[F]$
0	–	0	∞	∞	∞	∞	∞
1	A	0	4	2	∞	∞	∞
2	C	0	3	2	10	12	∞
3	B	0	3	2	8	12	∞
4	D	0	3	2	8	10	14
5	E	0	3	2	8	10	13
6	F	0	3	2	8	10	13

Thứ tự chốt: $A \rightarrow C \rightarrow B \rightarrow D \rightarrow E \rightarrow F$.

Vì sao cần Bellman–Ford?

Ý chính

Bellman–Ford xử lý được cạnh âm và có khả năng phát hiện chu trình âm đi tới được từ nguồn.

- Nếu không có chu trình âm đi tới được từ nguồn, mọi đường đi ngắn nhất đơn giản có nhiều nhất $|V| - 1$ cạnh.
- Vì vậy, lặp nối lỏng tất cả cạnh $|V| - 1$ lần là đủ để lan truyền khoảng cách ngắn nhất.
- Một lượt nối lỏng bổ sung dùng để kiểm tra chu trình âm.

Pseudocode Bellman–Ford

- ❶ Khởi tạo $d[s] = 0$, $d[v] = \infty$ với mọi $v \neq s$; đặt $\text{pred}[v] = \text{None}$.
- ❷ Lặp $|V| - 1$ lần:
 - ❶ Với mọi cạnh $(u, v) \in E$, nếu $d[v] > d[u] + w(u, v)$ thì cập nhật $d[v]$ và $\text{pred}[v]$.
- ❸ Kiểm tra thêm một lượt trên mọi cạnh (u, v) :
 - ❶ Nếu vẫn còn thể cải thiện $d[v]$, tồn tại chu trình âm đi tới được từ nguồn.

Độ phức tạp

Thời gian $O(|V||E|)$, bộ nhớ phụ $O(|V|)$ ngoài phần lưu đồ thị.

Diễn giải kết quả Bellman–Ford

- Nếu không phát hiện chu trình âm, các giá trị $d[v]$ là khoảng cách ngắn nhất từ s tới v .
- Nếu phát hiện chu trình âm, cần xác định các đỉnh bị ảnh hưởng bởi chu trình đó.
- Với các đỉnh bị ảnh hưởng, không có khoảng cách ngắn nhất hữu hạn vì có thể đi vòng để giảm chi phí vô hạn lần.

Phát biểu chính xác

Bellman–Ford không chỉ “dừng khi có cạnh âm”; nó dừng khi cần xử lý cạnh âm và cần phát hiện chu trình âm có thể đi tới từ nguồn.

Ví dụ Bellman–Ford có cạnh âm nhưng không có chu trình âm

Xét danh sách cạnh theo đúng thứ tự quét:

$$(A, B, -2), \quad (B, C, 3), \quad (S, A, 4), \quad (S, B, 5).$$

- Nguồn là S .
- Thứ tự quét cạnh ảnh hưởng đến tốc độ lan truyền cập nhật giữa các lượt, nhưng không làm thay đổi kết quả cuối nếu không có chu trình âm.

Khoảng cách ngắn nhất từ S

$$d[S] = 0, \quad d[A] = 4, \quad d[B] = 2, \quad d[C] = 5.$$

Bảng chạy Bellman–Ford theo từng vòng lặp

Với $|V| = 4$, ta có tối đa $|V| - 1 = 3$ lượt nói lỏng toàn bộ danh sách cạnh.

Lượt	Cập nhật xảy ra trong lượt	$d[S]$	$d[A]$	$d[B]$	$d[C]$
0	Khởi tạo	0	∞	∞	∞
1	$S \rightarrow A, S \rightarrow B$	0	4	5	∞
2	$A \rightarrow B, B \rightarrow C$	0	4	2	5
3	Không còn cải thiện	0	4	2	5
Kiểm tra	Không còn cạnh nào nói lỏng được	0	4	2	5

Vì lượt kiểm tra cuối không còn cải thiện, đồ thị không có chu trình âm đi tới được từ nguồn S .

Mục tiêu

Tính $\text{dist}(i, j)$ cho mọi cặp đỉnh (i, j) .

- Có thể chạy thuật toán một nguồn nhiều lần.
- Floyd–Warshall là quy hoạch động trực tiếp cho mọi cặp.
- Phù hợp với đồ thị nhỏ hoặc vừa, hoặc đồ thị dày cần truy vấn mọi cặp.

Đánh số các đỉnh là $1, 2, \dots, n$. Gọi $d^{(k)}[i][j]$ là khoảng cách ngắn nhất từ i đến j chỉ dùng các đỉnh trung gian trong tập $\{1, \dots, k\}$.

$$d^{(k)}[i][j] = \min \left\{ d^{(k-1)}[i][j], d^{(k-1)}[i][k] + d^{(k-1)}[k][j] \right\}.$$

- Khởi tạo $d[i][i] = 0$.
- Nếu có cạnh (i, j) thì $d[i][j] = w(i, j)$; nếu không có cạnh thì $d[i][j] = \infty$.

Floyd–Warshall: điều kiện và độ phức tạp

- Cho phép cạnh âm, nhưng kết quả khoảng cách hữu hạn chỉ có nghĩa khi không có chu trình âm liên quan tới cặp đỉnh đang xét.
- Sau thuật toán, nếu tồn tại $d[i][i] < 0$ thì có chu trình âm đi tới được từ i và quay lại i .
- Thời gian: $O(|V|^3)$.
- Bộ nhớ: $O(|V|^2)$ nếu chỉ cần ma trận khoảng cách.
- Muốn khôi phục đường đi, cần lưu thêm ma trận truy vết như `next` hoặc `pred`.

Ví dụ Floyd–Warshall theo các vòng k

Xét đồ thị có 3 đỉnh với các cạnh: $1 \rightarrow 2 = 4$, $1 \rightarrow 3 = 11$, $2 \rightarrow 3 = 3$, $3 \rightarrow 1 = 2$.

Khởi tạo $D^{(0)}$:

$$\begin{bmatrix} 0 & 4 & 11 \\ \infty & 0 & 3 \\ 2 & \infty & 0 \end{bmatrix}$$

Sau vòng $k = 1$ (cho phép dùng đỉnh 1 làm trung gian):

$$\begin{bmatrix} 0 & 4 & 11 \\ \infty & 0 & 3 \\ 2 & 6 & 0 \end{bmatrix}$$

Sau vòng $k = 2$:

$$\begin{bmatrix} 0 & 4 & 7 \\ \infty & 0 & 3 \\ 2 & 6 & 0 \end{bmatrix}$$

Hoàn tất ví dụ Floyd–Warshall

Tiếp tục với cùng ví dụ.

Sau vòng $k = 3$:

$$\begin{bmatrix} 0 & 4 & 7 \\ 5 & 0 & 3 \\ 2 & 6 & 0 \end{bmatrix}$$

Vòng	Cải thiện mới
$k = 1$	$d[3][2]$ giảm từ ∞ xuống 6 nhờ đường đi $3 \rightarrow 1 \rightarrow 2$
$k = 2$	$d[1][3]$ giảm từ 11 xuống 7 nhờ đường đi $1 \rightarrow 2 \rightarrow 3$
$k = 3$	$d[2][1]$ giảm từ ∞ xuống 5 nhờ đường đi $2 \rightarrow 3 \rightarrow 1$

Vì mọi phần tử đường chéo đều không âm, ví dụ này không có chu trình âm.

Chọn thuật toán phù hợp

Thuật toán	Điều kiện áp dụng	Độ phức tạp tiêu biểu
BFS	Mọi cạnh có trọng số đơn vị	$O(V + E)$
Dijkstra	Trọng số không âm	$O((V + E) \log V)$ với heap nhị phân
Bellman–Ford	Có thể có cạnh âm; phát hiện chu trình âm từ nguồn	$O(V E)$
Floyd–Warshall	Mọi cặp đỉnh; cho phép cạnh âm nhưng cần xét chu trình âm	$O(V ^3)$

Sai lầm thường gặp

- Dùng BFS cho đồ thị có trọng số không đồng nhất.
- Dùng Dijkstra khi có cạnh âm.
- Phát biểu thiếu điều kiện về chu trình âm trong Bellman–Ford hoặc Floyd–Warshall.
- Quên gán $d[s] = 0$ và $d[v] = \infty$ trước khi bắt đầu.
- Quên cập nhật $\text{pred}[v]$ khi nối lỏng cạnh nên không truy vết được đường đi.
- Không xử lý đỉnh không tới được từ nguồn.

Kiểm thử trường hợp biên

Mỗi cài đặt nên được kiểm thử trên các trường hợp sau:

- Đồ thị chỉ có một đỉnh.
- Đồ thị có đỉnh không tới được từ nguồn.
- Có nhiều đường đi cùng đạt tối ưu.
- Có cạnh trọng số 0.
- Có cạnh âm nhưng không có chu trình âm.
- Có chu trình âm đi tới được từ nguồn.

- 1 Tự lập bảng chạy cho BFS: hàng đợi, đỉnh lấy ra, các đỉnh mới phát hiện, và khoảng cách sau mỗi bước.
- 2 Với ví dụ Dijkstra trong bài, lập lại bảng “đỉnh chốt – vector khoảng cách” sau mỗi vòng lặp.
- 3 Cho một đồ thị có cạnh âm nhưng không có chu trình âm; lập bảng chạy Bellman–Ford theo từng lượt quét cạnh.
- 4 Với một ví dụ 4 đỉnh, viết các ma trận $D^{(0)}, D^{(1)}, \dots, D^{(n)}$ của Floyd–Warshall.

- Biểu diễn đồ thị bằng danh sách kề: $\text{adj}[u] = [(v, w), \dots]$.
- Dijkstra thường dùng `heapq`; cần bỏ qua bản ghi cũ nếu khóa trong heap không còn khớp với $d[u]$.
- Bellman–Ford nên lưu danh sách cạnh $\text{edges} = [(u, v, w), \dots]$.
- Floyd–Warshall dùng ma trận $n \times n$; cần cẩn thận với giá trị ∞ để tránh cộng tràn hoặc cộng vô nghĩa.

- Vì sao điều kiện trọng số không âm là đủ để Dijkstra đúng?
- Chu trình âm làm thay đổi ý nghĩa của “đường đi ngắn nhất” như thế nào?
- Với đồ thị rất thưa và chỉ cần một nguồn, thuật toán nào hợp lý nhất?
- Với đồ thị nhỏ nhưng cần trả lời nhiều truy vấn mọi cặp, nên tiền xử lý bằng thuật toán nào?

Thông điệp chính

Không có một thuật toán đường đi ngắn nhất tốt nhất cho mọi tình huống; lựa chọn đúng phụ thuộc vào loại trọng số, số lượng truy vấn và kích thước đồ thị.

- BFS: cạnh đơn vị.
- Dijkstra: trọng số không âm, hiệu quả trên đồ thị thưa.
- Bellman–Ford: cạnh âm và phát hiện chu trình âm từ nguồn.
- Floyd–Warshall: mọi cặp đỉnh, đồ thị nhỏ/vừa hoặc cần tiền xử lý toàn cục.

- Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, chương về single-source shortest paths và all-pairs shortest paths.
- Kleinberg, Tardos. *Algorithm Design*, phần graph algorithms.
- MIT 6.006: Dijkstra's Algorithm, Bellman–Ford, và shortest paths.
- NetworkX documentation: shortest path algorithms.